Explainable AI: Concepts, Techniques, and Implementations

From Taxonomy to Practical Methods (LIME, SHAP, Grad-CAM, and Beyond)

Previous session

- Need for XAI
- XAI Methods
  - PCI, Occlusion

Today's Session Overview

In today's session, we will explore:

- SHapley Additive exPlanations (SHAP): **Permutation-based method**
- Local Interpretable Model-Agnostic Explanations (LIME): **Post-hoc, Model Agnostic**

SHAP

For an image, the features are usually individual pixels or small groups of pixels, often called "superpixels" or "segments".

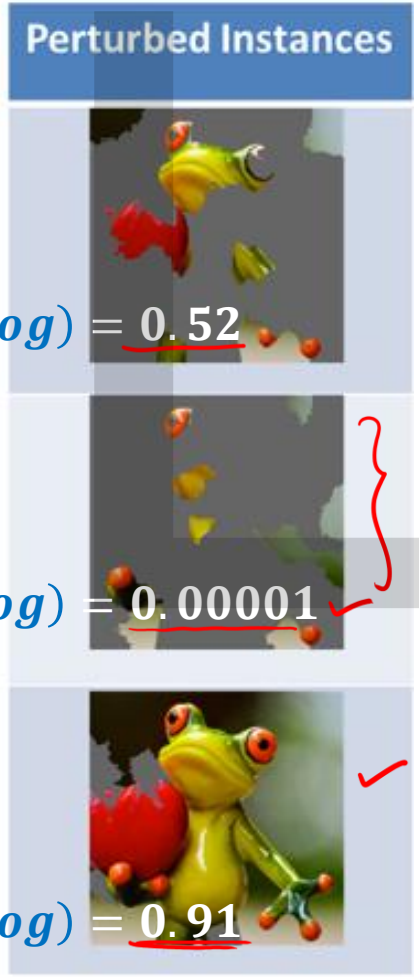
$P(tree\ frog) = 0.94$

Build a CNN model, train the model on original image, and get the model prediction

A superpixel is a **small region of the image** where all the pixels look similar. Instead of **treating every pixel separately**, we group similar pixels into **larger chunks.**
A single pixel has no meaning to a CNN (does not understand one pixel, but understands patterns).
Superpixels capture edges, shapes, textures, which matter more than single pixels.

SHAP explains which regions of an image influence the model's prediction.

- Divide the image into superpixels.
- Create perturbed versions:
  - Some superpixels ON (visible)
  - Some OFF (masked)
- Pass each version through the model.



Calculate Shapley Values: SHAP's core algorithm then calculates a Shapley value for each feature (pixel/segment).

This fraction tells how much importance to give to this particular subset S. How important this subset is.

ϕ_i is the **SHAP value** for feature **i**.
It tells: *"How much did feature i contribute to the final prediction?"*

$$\phi_i = \sum_{S \subseteq F\{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

weight

How much does the output change when we add feature i to S? This quantity is called the **marginal contribution** of feature i.

How much **one feature** contributes to the model's prediction.

For tabular data:
A *feature* is a column like:
- Age
- Blood Pressure
- Cholesterol
- Income

SHAP value tells how much each column contributed to the prediction.

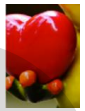
For image data:
A *feature* is **not a pixel**, but a **superpixel**.
So for image SHAP:
- **Each superpixel is treated as one feature**.
- SHAP computes how much that superpixel contributed to the model's prediction.

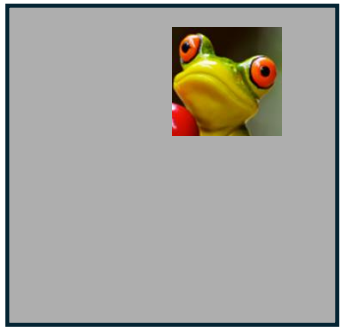
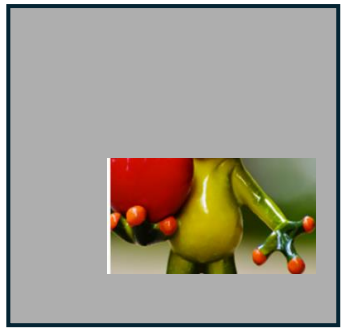
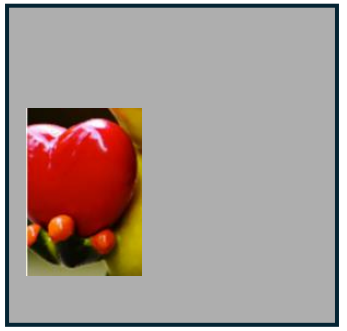
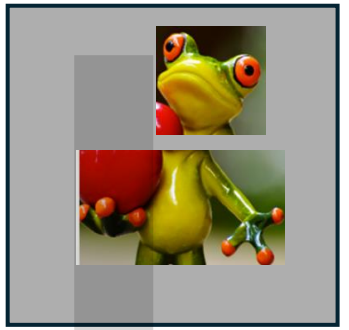
F: *Set of all features*
S: A *subset* of features that **does NOT include i**.

{A,B,C}

Example:
subsets S (if features = {A, B, C} and i = C):
$\{\}, \{A\}, \{B\}, \{A, B\}$

The image is divided into **3 superpixels**:

- Superpixel A (a region like the frog's face) 
- Superpixel B (another region like body) 
- Superpixel C (background area) 

The model's output is a probability (say, probability of "frog").

Subset of 2^n Superpixels ($2^3 = 8$)	Model Prediction $f(S)$
∅ (none kept)	0.1
{A} Only A	0.7
{B} Only B	0.5
{C} Only C	0.2
{A, B} Only A+B	0.75
{A, C} Only A+C	0.6
{B, C} Only B+C	0.45
{A, B, C}	0.9

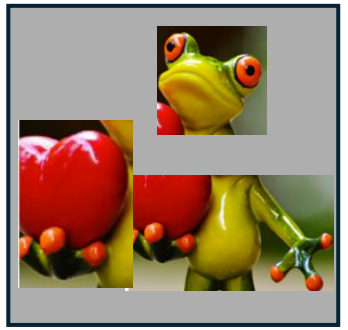
$$\phi_i = \sum_{S \subseteq F\{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

Here $F = \{A, B, C\}$,so $|F| = 3$.

The weights depend on the subset size:

- For $|S| = 0$ → weight = $\frac{0! \cdot 2!}{3!} = \frac{2}{6} = \frac{1}{3}$
- For $|S| = 1$ → weight = $\frac{1! \cdot 1!}{3!} = \frac{1}{6}$
- For $|S| = 2$ → weight = $\frac{2! \cdot 0!}{3!} = \frac{2}{6} = \frac{1}{3}$

$|S| : 3$

features = {A, B, C}
and i = A):
$\{\}, \{B\}, \{C\}, \{B, C\}$

$$\phi_i = \sum_{S \subseteq F\{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

$\|S\| = 0$	$\frac{1}{3}$
$\|S\| = 1$	$\frac{1}{6}$
$\|S\| = 2$	$\frac{1}{3}$

Subset of Superpixels	Model Prediction $f(S)$
∅ (none kept)	0.1
{A}	0.7
{B}	0.5
{C}	0.2
{A, B}	0.75
{A, C}	0.6
{B, C}	0.45
{A, B, C}	0.9

Superpixel A

$[f(S \cup \{A\}) - f(S)]$

When only Superpixel A is kept, the probability of predicting 'frog' increases by +0.6, weighted to +0.2.

S = ∅: weight = $1/3 * \left(f(A) - f(\emptyset)\right) = 0.7 - 0.1 = 0.6$ →contribution = +0.2

- S = {B}: weight = $1/6 * \left(f(A,B) - f(B)\right) = 0.75 - 0.5 = 0.25$ →contribution = +0.04166
- S = {C}: weight = $1/6 * \left(f(A,C) - f(C)\right) = 0.6 - 0.2 = 0.4$ →contribution = +0.0667
- S = {B,C}: weight = $1/3 * \left(f(A,B,C) - f(B,C)\right) = 0.9 - 0.45 = 0.45$ →contribution = +0.15

Total φ(A) = 0.2 + 0.04166 + 0.0667 + 0.15 = 0.45836 ≈ 0.46

When B is already present, adding A still increases the probability by 0.25 (weighted to 0.0417).

Superpixel A (frog's face) contributes **about +0.46** to the model's final prediction.

Superpixel B

features = {A, B, C}
and i = B):
$\{\}, \{A\}, \{C\}, \{A, C\}$

$[f(S \cup \{B\}) - f(S)]$

S = ∅: weight = $1/3 * \left(f(B) - f(\emptyset)\right) = 0.5 - 0.1 = 0.4$ → contribution = +0.1333

- S = {A}: weight = $1/6 * \left(f(A,B) - f(A)\right) = 0.75 - 0.7 = 0.05$ → contribution = +0.0083
- S = {C}: weight = $1/6 * \left(f(B,C) - f(C)\right) = 0.45 - 0.2 = 0.25$ → contribution = +0.04166
- S = {A,C}: weight = $1/3 * \left(f(A,B,C) - f(A,C)\right) = 0.9 - 0.6 = 0.3$ → contribution = +0.1

Total φ(B) = 0.1333 + 0.0083 + 0.04166 + 0.1 = 0.28326 ≈ 0.28 ✓

features = {A, B, C}
and i = C):
$\{\}, \{A\}, \{B\}, \{A, B\}$

$$\phi_i = \sum_{S \subseteq F\{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

$\lvert S\rvert = 0$	$\frac{1}{3}$
$\lvert S\rvert = 1$	$\frac{1}{6}$
$\lvert S\rvert = 2$	$\frac{1}{3}$

Subset of Superpixels	Model Prediction $f(S)$
∅ (none kept)	0.1
{A}	0.7
{B}	0.5
{C}	0.2
{A, B}	0.75
{A, C}	0.6
{B, C}	0.45
{A, B, C}	0.9

Superpixel C $[f(S \cup \{C\}) - f(S)]$

S = ∅: weight = $1/3 * \big(f(C) - f(\emptyset)\big)$ = 0.2 - 0.1 = 0.1 → contribution = +0.0333

•S = {A}: weight = $1/6 * \big(f(A,C) - f(A)\big)$ = 0.6 - 0.7 = -0.1 → contribution = -0.0166

•S = {B}: weight = $1/6 * \big(f(B,C) - f(B)\big)$ = 0.45 - 0.5 = -0.05 → contribution = -0.00833

•S = {A,B}: weight = $1/3 * \big(f(A,B,C) - f(A,B)\big)$ = 0.9 - 0.75 = 0.15 → contribution = +0.05

Total φ(C) = 0.0333 - 0.0166 – 0.00833 + 0.05 = 0.05837 ≈ 0.06

Check Consistency

Because SHAP values must always satisfy an **important property of cooperative game theory**:
The contributions of all features must add up exactly to the model output difference.
This is called the **efficiency axiom** of Shapley values.

$$\phi_A + \phi_B + \phi_C = f(A,B,C) - f(\emptyset)$$

$$0.46 + 0.28 + 0.06 = 0.90 - 0.10 = 0.80$$

The total model prediction must be **distributed fully** among the features

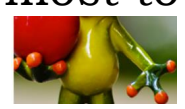
Final Shapley Values

•φ(A) ≈ **0.46**
•φ(B) ≈ **0.28**
•φ(C) ≈ **0.06**

This **0.80 is the total influence** of all three superpixels **combined**. SHAP now needs to **distribute** this total influence among A, B, and C **fairly**.

✔ **0.80 is the total effect that must be explained by SHAP.**

Interpretation: Superpixel A contributes most to the model's prediction, followed by B, then C.



Heatmap:

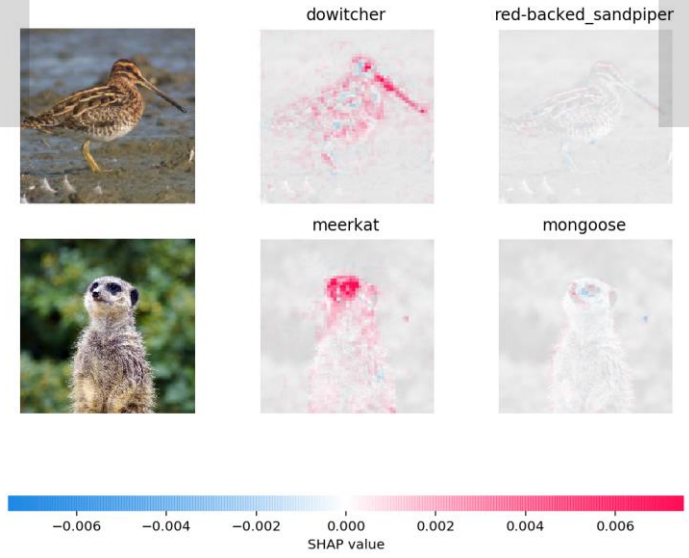
A **superpixel** is treated as **one feature**.
The Shapley value for superpixel A is:

$$\phi(A) = +0.46$$

then **every pixel** inside that superpixel gets the **same SHAP value: +0.46**.

+0.46 is a large positive value. Large positive values are colored strong red.

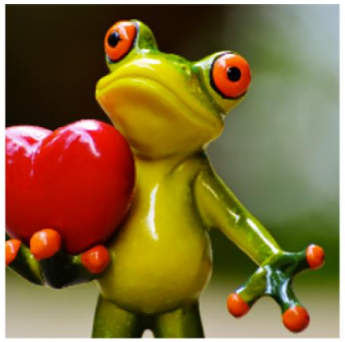
So, the entire region belonging to superpixel A becomes bright / strong red in the heatmap.

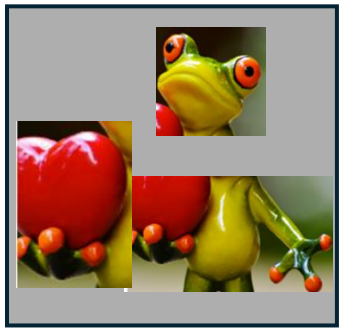


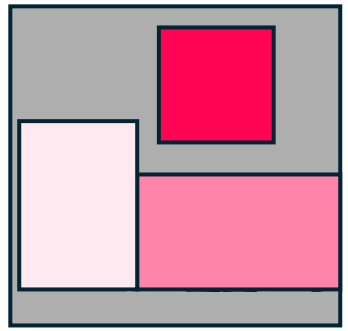
Final Shapley Values

- φ(A) ≈ **0.46**
- φ(B) ≈ **0.28**
- φ(C) ≈ **0.06**

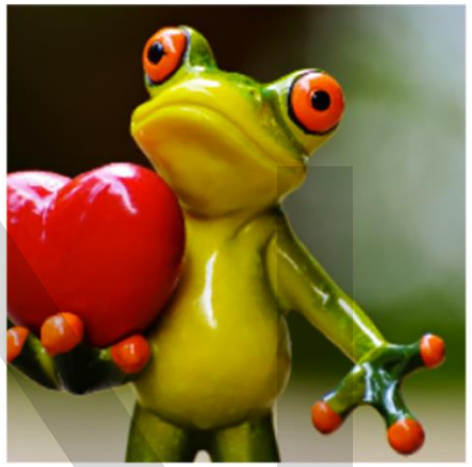
✔ **Superpixel A → +0.46 → Strong Red**
→ Because it strongly supports the "frog" prediction.
✔ **Superpixel B → +0.28 → Medium Red**
→ Supports the prediction but less strongly.
✔ **Superpixel C → +0.06 → Very Light Red / Almost White**
→ Has very small positive effect.







LIME



Each superpixel is treated as a **binary feature**:

1 → superpixel is **kept** (original)

0 → superpixel is **turned off** (usually replaced by grey/blur/mean color)

So, an image with 50 superpixels becomes a vector of 50 binary values.

A ON, B OFF, C ON → (1,0,1)

A OFF, B OFF, C ON → (0,01)

(0,0,1)

$P(tree\ frog) = 0.52$

$P(tree\ frog) = 0.00001$

$P(tree\ frog) = 0.91$

0.91

The image is divided into **3 superpixels**:

- Superpixel A (a region like the frog's face) 
- Superpixel B (another region like body) 
- Superpixel C (background area) 

The model's output is a probability (say, probability of "frog").

perturbed samples Original: (1,1,1) – All the Superpixel are kept

Superpixels kept	A	B	C	Model prob (f(S))	Difference	Hamming	Euclidean (D)
none	0	0	0	0.10	(1,1,1)	3	$\sqrt{(1^2+1^2+1^2)} = 1.732$
{A}	1	0	0	0.70	(0,1,1)	2	$\sqrt{(0^2+1^2+1^2} = 1.414$
{B}	0	1	0	0.50	(1,0,1)	2	$\sqrt{(1^2+0^2+1^2} = 1.414$
{C}	0	0	1	0.20	(1,1,0)	2	$\sqrt{(1^2+1^2+0^2} = 1.414$
{A,B}	1	1	0	0.75	(0,0,1)	1	$\sqrt{(0^2+0^2+1^2} = 1$
{A,C}	1	0	1	0.60	(0,1,0)	1	$\sqrt{(0^2+1^2+0^2} = 1$
{B,C}	0	1	1	0.45	(1,0,0)	1	$\sqrt{(1^2+0^2+0^2} = 1$
{A,B,C}	1	1	1	0.90	(0,0,0)	0	$\sqrt{(0^2+0^2+0^2} = 0$

$$W = \exp\left(-\frac{D^2}{\sigma^2}\right)$$

σ is the kernel width used in LIME's *exponential kernel*

$$\sigma = 0.75\sqrt{d} \quad where\ d = number\ of\ features$$
$$= 0.75\sqrt{3}$$
$$= 0.75 * 1.732$$
$$= 1.299$$
$$\sigma^2 = 1.299^2 = 1.6874$$

Superpixels kept	A	B	C	Euclidean (D)	Weight
none	0	0	0	1.732	0.169
{A}	1	0	0	1.414	0.306
{B}	0	1	0	1.414	0.306
{C}	0	0	1	1.414	0.306
{A,B}	1	1	0	1	0.553
{A,C}	1	0	1	1	0.553
{B,C}	0	1	1	1	0.553
{A,B,C}	1	1	1	0	1

This kernel gives **higher weight** to samples that are **closer** to the original instance and **lower weight** to samples that are **far away.**

For D=1.732

$$D^2 = 1.732^2 = 2.999$$
$$\frac{D^2}{\sigma^2} = \frac{2.999}{1.687} = 1.777$$
$$W = e^{-1.777} = 0.169$$

For D=1.414

$$D^2 = 1.414^2 = 2$$
$$\frac{D^2}{\sigma^2} = \frac{2}{1.687} = 1.185$$
$$W = e^{-1.185} = 0.3055$$

For D=1

$$D^2 = 1 = 1$$
$$\frac{D^2}{\sigma^2} = \frac{1}{1.687} = 0.5926$$
$$W = e^{-0.5926} = 0.5528$$

For D=0

$$W = e^{0} = 1$$

weighted linear regression

Quick revision of Multi Linear Regression

Person	Age (x_1)	Experience (x_2)	Salary (y in ₹ Lakhs/year)
1	22	0	3.0
2	25	2	4.2
3	28	4	5.5
4	30	6	6.2
5	32	7	7.0
6	35	10	8.5
7	40	15	11.0
8	45	20	13.0
9	50	25	15.0
10	55	30	17.0

Because multi linear regression tries to model the relationship between x_1, x_2 and y using the simplest possible function — a *straight line*.

$$\hat{y} = \beta_o + \beta_1 x_1 + \beta_2 x_2$$

The best values of $\beta_0, \beta_1, \beta_2$ describe how x_1, x_2 and y are related.

How does the regression find the best $\beta_0, \beta_1, \beta_2$

By minimizing the total error:

$$Error = \sum_{i=1}^{10} (y - \hat{y})^2$$

So the algorithm searches for the values of $\beta_0, \beta_1, \beta_2$ that make predictions $\hat{y}$ closest to actual y.

Gradient Descent is used whenever we want to **find the best parameters** that minimize a loss/error function.

- If we **increase** β_1 ,does error increase or decrease?
- If we **decrease** β_1 ,does error increase or decrease?

It computes **slopes** (gradients):

$$\frac{\partial Error}{\partial \beta_0}, \frac{\partial Error}{\partial \beta_1}, \frac{\partial Error}{\partial \beta_2}$$

$$\beta_0 = \beta_0 - \eta \frac{\partial Error}{\partial \beta_0}$$

$$\beta_1 = \beta_1 - \eta \frac{\partial Error}{\partial \beta_1}$$

$$\beta_2 = \beta_2 - \eta \frac{\partial Error}{\partial \beta_2}$$

i	Superpixels kept	Features			Target	distance D	weight w_i
		A	B	C	$y_i = f(S)$		
1	none	0	0	0	0.10	1.732	0.169
2	{A}	1	0	0	0.70	1.414	0.306
3	{B}	0	1	0	0.50	1.414	0.306
4	{C}	0	0	1	0.20	1.414	0.306
5	{A,B}	1	1	0	0.75	1.000	0.553
6	{A,C}	1	0	1	0.60	1.000	0.553
7	{B,C}	0	1	1	0.45	1.000	0.553
8	{A,B,C}	1	1	1	0.90	0.000	1.000

weighted linear regression

$LIME's\ surrogate\ model$: $\hat{y}_i = \beta_0 + \beta_A A_i + \beta_B B_i + \beta_C C_i$

The weighted loss is:

$$Loss(\beta_0, \beta_A, \beta_B, \beta_C) = \sum_{i=1}^{8} w_i (y_i - \hat{y}_i)^2 = \sum_{i=1}^{8} w_i (y_i - (\beta_0 + \beta_A A_i + \beta_B B_i + \beta_C C_i))^2$$

Goal: **choose $\beta_0, \beta_A, \beta_B, \beta_C$ that make this Loss as small as possible.**

$$\beta_0 \approx 0.18,$$
$$\beta_A \approx 0.41,$$
$$\beta_B \approx 0.24,$$
$$\beta_C \approx 0.04$$

So, the LIME surrogate model for this frog image is:

$$\hat{y} = 0.18 + 0.41A + 0.24B + 0.04C$$

Now the **LIME explanation** is just these βvalues:

- **Superpixel A (frog face)**
  $\beta_A \approx 0.41$ →strong positive influence

- **Superpixel B (body + heart)**
  $\beta_B \approx 0.24$ →moderate positive influence

- **Superpixel C (only heart)**
  $\beta_C \approx 0.04$ →very small positive influence

Step 1: Segment the image into superpixels

- Break the input image into meaningful regions (superpixels).
- Each superpixel becomes a binary feature:
  - 1 → keep the region ✓
  - 0 → turn it off (mask it) ✓
- If the image produces d superpixels, then each perturbed sample is a d-dimensional binary vector.

Step 2: Generate perturbed samples

- Create thousands of new images by randomly turning different superpixels ON or OFF.
- Each perturbed image corresponds to a binary vector like:

$$(1,0,1,1,0,0,\ldots.) \checkmark$$

- When OFF, the region is replaced by grey/blur/mean color.

Step 3: Get predictions from the black-box model

- Feed every perturbed image to the original classifier (CNN).
- Record the output probability for the target class

$$y_i = f(S_i) \checkmark$$

Step 4: Compute distance between each perturbed sample and the original

- Compute distance in binary space (usually **Euclidean**) between:
  - Original mask: (1,1,1,…,1) – All features kept
  - Perturbed mask: (1,0,1,…,1) - some features masked

Example distance:

$$D_i = \sqrt{(1-1)^2+(1-0)^2+(1-1)^2+\cdots+(1-1)^2}$$

Step 5: Calculate the weights ✓

Use the kernel function:

$$w_i = \exp\left(-\frac{D_i^2}{\sigma^2}\right)$$

Where,

$$\sigma = 0.75\sqrt{d},$$
$$where\ d = number\ of\ features$$

D_i	Perturbation	Weight	σ
Small	Very close	High ✓	Small
Large	Far away	Low weight ✓	Large

Step 6: Fit a weighted linear regression (local surrogate model)

- LIME now fits a simple linear model:

$$\hat{y}(z) = \beta_0 + \beta_A A + \beta_B B + \beta_C C$$

- The weighted loss is:

$$Loss = \sum_{i=1}^{N} w_i(y_i - \hat{y}_i)^2$$

- The regression finds β-values for each superpixel.
- These β-values tell how important each superpixel is for the prediction.
- Higher $\beta \Rightarrow$ stronger influence on prediction.

Step 7: Visualize the explanation

LIME can able to give explanations for the following type of datasets:

1.Tabular datasets *(lime.lime_tabular.LimeTabularExplainer)* eg: Regression, Classification datasets

2.Image-related datasets *(lime.lime_image.LimeImageExplainer)*

3.Text-related datasets *(lime.lime_text.LimeTextExplainer)*

SHAP can able to give explanations for the following type of datasets:

1. Tabular datasets *(shap.TreeExplainer, shap.KernelExplainer, shap.LinearExplainer)*

eg: Regression, Classification datasets

2. Image-related datasets *(shap.DeepExplainer, shap.GradientExplainer, shap.KernelExplainer)*

3. Text-related datasets *(shap.TextExplainer, shap.KernelExplainer)*

Next Session

- Guided backpropagation

Thank you